1. Time Complexity Calculations of Algorithms

1) Algorithm 1: Receiver Knapsack Key Generation

Let c_i be the constant computational time required for a fundamental atomic operation, k be the expected number of iterations required to randomly choose a prime number, and E be the time required to calculate the Euclidean algorithm for the Greatest Common Divisor (GCD).

Line	Operation	Cost	Frequency
1	Initialize PR_{rx}	c_1	N
2	Compute TotalSum	c_2	N
3	Generate Modulus M_{rx}	c_3	1
4-6	REPEAT… UNTIL gcd(N_{rx}, M_{rx}) == 1	c_4 + E	k
9-11	FOR i = 0 to length(PR_{rx}) – 1 DO	c_5	N
10	$PU_{rx}[i] \leftarrow (PR_{rx}[i] * N_{rx}) \bmod (M_{rx})$	c_6	N
12	RETURN PU_{rx}, PR_{rx}, M_{rx}, N_{rx}	c_7	1

Table 1 – Table method for considering time complexity of Algorithm 1

From the table 1, the following initial time complexity equation is obtained:

$$T(N) = c_1N + c_2N + c_3 + k(c_4 + E) + c_5N + c_6N + c_7$$

Since the Euclidean algorithm calculates the GCD in logarithmic time, E = O(log M_{rx}). Euler's Totient function influences that the probability of randomly hitting a coprime is high as primes and coprimes are densely distributed. Thus, the expected number of trials (k) is evaluated to a small, mathematically bounded constant and hence, and the entire REPEAT … UNTIL block addresses to a computational constant C_{gcd} that does not scale with N.

$$T(N) = N(c_1 + c_2 + c_5 + c_6) + (c_3 + C_{gcd} + c_7)$$

The mathematical induction is applied to prove that the public key transformation (FOR loop in line 9-11) needs exactly O(N) operations. Let P(N) be the proposition that states that the transformation loop executes exactly N times, and bounded linear time is required for that execution. If the key array size if N = 1, the loop starts its iteration from i = 0 to 0 and executes only a single time exactly and thus P(1) holds true. Assume that P(m) is also true which means

that for an array of size m, the inductive hypothesis states that the loop exactly executes m times, evaluating in O(m) time. The inductive step needs to prove that P(m + 1) is true.

For an array of size m + 1, the loop iterates from i = 0 to m. The iteration can be split into the iterations from i = 0 to m – 1 which is exactly m iterations by the hypothesis in addition to the final execution where i = m which is exactly 1 additional iteration.

$$T(m+1) = T(m) + 1 = m + 1$$

Since P(m + 1) holds, by the principle of mathematical induction, it is proved that the transformation loop executes strictly O(N) time for all N ≥ 1.

When the constant operational coefficients ($c_1 + c_2 + c_3 + c_4$) are factored out and the non-scaling procedural constants ($c_3 + C_{gcd} + c_7$) are removed from the initial equation, the highest-order scaling term N is achieved.

$$T(N) = O(N)$$

2) Algorithm 2: Sender Knapsack Key Generation

Let c_i be the constant computational time required for a fundamental atomic operation, k be the expected number of iterations required to randomly choose a prime number, and E be the time required to calculate the Euclidean algorithm for the Greatest Common Divisor (GCD).

Line	Operation	Cost	Frequency
1	Initialize PR_{tx}	c_1	N
2	Compute TotalSum	c_2	N
3	Generate Modulus M_{tx}	c_3	1
4-6	REPEAT… UNTIL gcd(N_{tx}, M_{tx}) == 1	$c_4 + E$	k
9-11	FOR i = 0 to length(PR_{tx}) – 1 DO	c_5	N
10	$PU_{tx}[i] \leftarrow (PR_{tx}[i] * N_{tx}) \bmod (M_{tx})$	c_6	N
12	RETURN PU_{tx}, PR_{tx}, M_{tx}, N_{tx}	c_7	1

Table 2 – Table method for considering time complexity of Algorithm 2

From the table 2, the following initial time complexity equation is obtained:

$$T(N) = c_1N + c_2N + c_3 + k(c_4 + E) + c_5N + c_6N + c_7$$

Since the Euclidean algorithm calculates the GCD in logarithmic time, E = O(log M_{rx}). Euler's Totient function influences that the probability of randomly hitting a coprime is high as primes and coprimes are densely distributed. Thus, the expected number of trials (k) is evaluated to a small, mathematically bounded constant and hence, and the entire REPEAT … UNTIL block addresses to a computational constant C_{gcd} that does not scale with N.

$$T(N) = N(c_1 + c_2 + c_5 + c_6) + (c_3 + C_{gcd} + c_7)$$

The mathematical induction is applied to prove that the public key transformation (FOR loop in line 9-11) needs exactly O(N) operations. Let P(N) be the proposition that states that the transformation loop executes exactly N times, and bounded linear time is required for that execution. If the key array size if N = 1, the loop starts its iteration from i = 0 to 0 and executes only a single time exactly and thus P(1) holds true. Assume that P(m) is also true which means that for an array of size m, the inductive hypothesis states that the loop exactly executes m times, evaluating in O(m) time. The inductive step needs to prove that P(m + 1) is true.

For an array of size m + 1, the loop iterates from i = 0 to m. The iteration can be split into the iterations from i = 0 to m – 1 which is exactly m iterations by the hypothesis in addition to the final execution where i = m which is exactly 1 additional iteration.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since P(m + 1) holds, by the principle of mathematical induction, it is proved that the transformation loop executes strictly O(N) time for all N ≥ 1.

When the constant operational coefficients ($c_1 + c_2 + c_3 + c_4$) are factored out and the non-scaling procedural constants ($c_3 + C_{gcd} + c_7$) are removed from the initial equation, the highest-order scaling term N is achieved.

$$T(N) = O(N)$$

3) Algorithm 3: Chaotic Parameter Initialization

Let c_i be the constant computational time needed for a fundamental atomic operation, and N be the architectural length of the binary session vector $V_{session}$. Let P be the expected number of procedurals retries needed to generate a vector whose sum is strictly beyond zero.

Line	Operation	Cost	Frequency
1-2	Generate Inner Shift C_1, C_2	c_1	1
3-5	Generate L_x, L_y, L_z	c_2	1
6-12	REPEAT	0	0
7	Initialize $V_{session}$ array	c_3	P
8-10	FOR i = 0 to N – 1 DO	c_4	P . N
9	$V_{session}[i] \leftarrow$ random binary bit	c_5	P . N
11	VectorSum ← Sum($V_{session}$)	c_6	P . N
12	UNTIL VectorSum > 0	c_7	P
13	RETURN $V_{session}$, C_1, C_2, L_x, L_y, L_z	c_8	1

Table 3 – Table method for considering time complexity of Algorithm 3

From the table 3, the following initial time complexity equation is obtained:

$$T(N) = c_1 + c_2 + P(c_3 + c_4N + c_5N + c_6N + c_7) + c_8$$

The REPEAT…UNTIL loop block restarts if the generated session vector has zeros entirely. Since each binary digit has 50% probability of being a 0 or 1, the probability of generating a completely zero-state vector of length N is 2^{-N}. Thus, the expected number of iterations P execution is:

$$P = \frac{1}{1 - 2^{-N}}$$

Since the architectural vector length is scaled up to the cryptographic standard of N = 256, the probability of generating the vector that has zeros entirely is extremely close to 0 which means this system never generates the zero only filled binary session vectors. As a result, the probability function acts as a constant function O(1) and it does not scale up to the exponential time complexity.

$$T(N) = N(c_4 + c_5 + c_6) + (c_1 + c_2 + c_3 + c_7 + c_8)$$

In order to prove that the summing an array of N binary elements performs in linear time, that is to prove the strict bounds of line 13, the mathematical induction is applied. Let P(N) be the proposition stating that calculating the total sum of an array that includes N elements requires N – 1 addition operations. As a base case, if the array contains exactly 1 element (N = 1), then there is

no addition operations required to process because the sum is the element itself which means P(1) is true. For inductive hypothesis, assume that P(m) is true meaning the summation of an array of m elements requires exactly m – 1 addition, evaluating in O(m) time. Then, the inductive step P(m + 1) is proved for the correctness.

For an array of size m + 1, the system calculates the sum of the first m elements which requires m – 1 additions by the hypothesis and performs exactly 1 final addition to include the (m + 1)th element.

$$(m - 1) + 1 = m$$

Since m = (m + 1) – 1, by the principle of mathematical induction, it is true that the summation assesses in linear O(N) operations.

When the constant operational coefficients ($c_4 + c_5 + c_6$) are factored out and the scalar allocation constants O(1) are dropped from the initial equation, the highest-order term enforcing the execution time is bounded by N.

$$T(N) = O(N)$$

4) Algorithm 4: Session Vector Encapsulation

Let c_i be the constant computational time necessary for executing a fundamental atomic operation. Then, the procedural execution flow of subset-sum encapsulation is evaluated.

Line	Operation	Cost	Frequency
1	Initialize $H_{enc} \leftarrow 0$	c_1	1
3-5	FOR i = 0 to length($V_{session}$) – 1 DO	c_2	N
4	$H_{enc} \leftarrow H_{enc} + (V_{session}[i] * PU_{rx}[i])$	c_3	N
7	SignatureTargetHash $\leftarrow H_{enc}$	c_4	1
8	RETURN H_{enc}	c_5	1

Table 4 – Table method for considering time complexity of Algorithm 4

From table 4, the following initial time complexity equation is obtained:

$$T(N) = c_1 + c_2N + c_3N + c_4 + c_5$$

The linear algebraic equation for the algorithm's execution time can be established by grouping the scaling and non-scaling terms together:

$$\mathrm{T(N)} = N(c_2 + c_3) + (c_1 + c_4 + c_5)$$

The mathematical induction is applied to prove that computing the dot product, which is line 5, of two one-dimensional arrays of size N in strict linear time in order to rigorously show the bounds of the core encapsulation step. Let P(N) be the proposition states that the dot product calculation of an N-dimensional vector requires N iterations exactly, executing in bounded linear time. The base case expresses that if the loop executes from i = 0 to 0 if the vectors are one-dimensional (i.e., N = 1). It iterates one times exactly which requires one multiplication and one addition. Thus, P(1) mathematically holds true. In it, induction hypothesis states that assuming P(m) is true meaning that the dot product of two vectors of size m needs m iterations exactly, evaluating in O(m) execution time. Then, the statement P(m + 1) is true is proved as a inductive step.

The algorithm calculates the dot product of the first m elements for vectors that have the size of m + 1 which needs m iterations by the inductive hypothesis, and performs one final iteration exactly to compute and add the product of (m + 1)th coordinates.

$$\mathrm{T(m + 1) = T(m) + 1 = m + 1}$$

Since m = (m + 1) – 1, P(m + 1) true. Therefore, by the principle of mathematical induction, it is true that the dot product loop evaluates in linear time O(N) operations for any vector length $N \geq 1$.

The highest-order scaling term that determines the algorithm's runtime is N after factoring out the constant loop operational coefficients ($c_2 + c_3$) and eliminating the O(1) scalar allocation and return constants ($c_1 + c_4 + c_5$) from the initial equation.

$$\mathrm{T(N) = O(N)}$$

5) Algorithm 5: Plaintext Cleaning and Format Extraction

Let c_i be the constant computational time needed for a fundamental atomic operation. The plaintext preparation algorithm workflow is evaluated for assessing time complexity.

Line	Operation	Cost	Frequency
1	ConvertNumbersToWords(RawText)	c_1	N
2	ToUpperCase(RawText)	c_2	N
3	Initialize Cleantext	c_3	1
4	Initialize MarkerMap	c_4	1
5-13	FOR i = 0 to length(RawText) – 1 DO	c_5	N
6	CurrentChar ← RawText[i]	c_6	N
7	IF CurrentChar in [‘A’ … ‘Z’]	c_7	N
8	CleanText ← CleanText + CurrentChar (Assuming optimal struct)	c_8	N (Worst-case)
10	Create Object (Index: i, Character)	c_9	N (Worst-case)
11	MarkerMap.Add(NewMarker)	c_{10}	N (Worst-case)
14	RETURN CleanText, MarkerMap	c_{11}	1

Table 5 – Table method for considering time complexity of Algorithm 5

The initial time complexity equation is derived from the table 5 in the following form:

$$T(N) = c_1N + c_2N + c_3 + c_4 + c_5N + c_7N + max\big(c_8N, (c_9N + c_{10}N)\big) + c_{11}$$

From line 9 to 14, the conditional code block executes either the THEN block, which is line 10, or the ELSE block, which is line 12 and 13. The most computationally expensive branch is assumed to be take for all N iterations for worse-case asymptotic bounds. Let $C_{branch} = \max(c_8, c_9 + 10_{10})$.

$$T(N) = N(c_1 + c_2 + c_5 + c_6 + c_7 + C_{branch}) + (c_3 + c_4 + c_{11})$$

In order to prove that the evaluation loop executes in linear time relative to the text length N, the mathematical induction is applied for strictly establishing the bounds of the main string processing method. Let P(N) be the proposition stating that it takes exactly N iterations to evaluate and route the characters in a text string of length N, and it can be executed in bounded linear time. For the base case, the loop iterates from i = 0 to 0 if the length of plaintext is N = 1. It means the

system processes exactly 1 character and performs one conditional IF statement evaluation and routing to the appropriate data structure. This makes the base case P(1) mathematically true. Then the inductive hypothesis assumes P(m) is true which means processing a string of m characters needs m loop iterations exactly, and it also evaluates in O(m) execution time. Then, the inductive step is applied to prove P(m + 1) is true.

The algorithm evaluates the first m characters which requires m iterations by the inductive hypothesis and performs exactly one final iteration to evaluate for a string of m + 1 characters, and append the (m + 1)th character.

$$T(m + 1) = T(m) + 1 = m + 1$$

It is obvious that P(m + 1) mathematically holds since m = (m + 1) – 1. Therefore, by the principle of mathematical induction, it is true that the character sorting loop evaluates in linear operations O(N) for any text length that has at least 1 or more than 1.

The highest-order scaling term for considering the algorithm's runtime is bounded by N by factoring out the constant loop operational coefficients ($c_1 + c_2 + c_5 + c_6 + c_7 + C_{branch}$) and dropping the O(1) initialization and return constants ($c_3 + c_4 + c_{11}$) from the initial table method equation.

$$T(N) = O(N)$$

6) Algorithm 6: First Shift and Alphabet Substitution

Let c_i be the constant computational time needed to execute a fundamental atomic operation. The algorithm's procedural workflow is evaluated using table method.

Line	Operation	Cost	Frequency
1-2	Initialize ShiftedText, MappedArray	c_1	1
4-8	FOR i = 0 to length(CleanText) – 1 DO	c_2	N
5	CharIndex ← CleanText[i] – 'A'	c_3	N
6	ShiftedIndex ← (CharIndex + C_1) mod 26	c_4	N
7	ShfitedText ← ShiftedText + (…)	c_5	N
10-15	FOR i = 0 to length(ShiftedText) – 1 DO	c_6	N
11	CurrentChar ← ShiftedText[i]	c_7	N
13	Lookup (EnochianMap, CurrentChar)	c_8	N
14	MappedArray.Append(…)	c_9	N
16	RETURN MappedArray	c_{10}	1

Table 6 – Table method for considering time complexity of Algorithm 6

The following initial time complexity equation is obtained from the table 6:

$$T(N) = c_1 + c_2N + c_3N + c_4N + c_5N + c_6N + c_7N + c_8N + c_9N + c_{10}$$

Since ShiftedText has the exact same length as CleanText, both sequential FOR loops iterate exactly N times. The operations are grouped according to their respective loops:

$$T(N) = N(c_2 + c_3 + c_4 + c_5) + N(c_6 + c_7 + c_8 + c_9) + (c_1 + c_{10})$$

Let C_{shift} be the constants that group the first loop, and C_{map} be the constants that group the second loop:

$$T(N) = N\left(C_{shift} + C_{map}\right) + (c_1 + c_{10})$$

The sequential dual-pass processing is proved by the mathematical induction. It aims to prove that the execution of two separate, un-nested loops over an array of size N is evaluated in linear time strictly. Let P(N) be the proposition states that the two sequential, distinct loops processing an array of size N that require 2N total iterations exactly that are executing in bounded linear time. The base case states that the first loop (shift) executes exactly 1 iteration if the input

text length is N = 1 and the second loop (substitution) subsequently executes exactly 1 iteration and there are total 2(1) = 2 iterations in one single process. Thus, this makes P(1) mathematically true. The following inductive hypothesis states that the P(m) is assume to be true meaning that processing of m characters through two sequential loops needs exactly 2m total iterations, evaluating in O(m) execution time. Them, the inductive step P(m + 1) is proved for its correctness.

Both the first and second loops process m + 1 characters for a string that includes m + 1 amount of characters. This can be expressed as the processing of the first m characters that is requiring 2m iterations by the inductive hypothesis in addition to 1 additional iteration in the first loop and 1 additional iteration in the second loop for the (m + 1)th character.

$$T(m + 1) = 2m + 1 + 1 = 2m + 2 = 2(m + 1)$$

Since P(m + 1) logically maps to the proposition, by the principle of mathematical induction, it is true that the dual-loop process evaluates in O(N) operations for any text length $N \geq 1$.

By factoring out the C_{shift} and C_{map} operational constants and dropping the O(1) initialization and return constants from the table method initial equation, the execution time is dominated by the highest-order scaling term N.

$$T(N) = O(N)$$

7) Algorithm 7: Second Shift and Matrix Allocation

Let c_i be the constant computational time needed to execute an atomic operation within a code block. The workflow of second shift and matrix allocation algorithm is evaluated using table method.

Line	Operation	Cost	Frequency
2-6	FOR i = 0 to length(MappedArray) – 1 DO	c_1	L
3-5	Split, Modulo Shift, Reassign	c_2	L
8-11	Initialize list, Capacity, Index	c_3	1
12	WHILE index < length(MappedArray)	c_4	$\left\lceil \frac{L}{N^2} \right\rceil$
13-14	Create Temp_Value_Matrix, Temp_Marker	c_5	$\left\lceil \frac{L}{N^2} \right\rceil$
15-27	Nested FOR loops (row and col)	c_6	$\left\lceil \frac{L}{N^2} \right\rceil . N^2$
17-25	IF/ELSE Allocation and Padding	c_7	$\left\lceil \frac{L}{N^2} \right\rceil . N^2$
28-29	ValueList.Append, MarkerList.Append	c_8	$\left\lceil \frac{L}{N^2} \right\rceil$
31	RETURN ValueList, MarkerList	c_9	1

Table 7 – Table method for considering time complexity of Algorithm 7

Let B be the total number of matrix blocks generated, where $B = \left\lceil \frac{L}{N^2} \right\rceil$. The following initial time complexity equation is derived from the table 7:

$$T(L) = L(c_1 + c_2) + c_3 + B(c_4 + c_5 + c_8) + B.N^2(c_6 + c_7) + c_9$$

The term B . N^2 represents the total number of matrix elements processed by the nested FOR loops. Mathematically:

$$B\,.N^2 = \left\lceil \frac{L}{N^2} \right\rceil . N^2$$

Since $\left\lceil \frac{L}{N^2} \right\rceil < \left(\frac{L}{N^2}\right) + 1$, The boundary can be expanded to:

$$B\,.N^2 < L + N^2$$

The Enochian system architecture states that a maximum constant of 10 strictly bounds the matrix dimension and hence $N^2 \leq 100$. The total number of inner loop iteration is strictly bounded by L + 100 because N^2 is an immutable architectural constant that does not scale with the payload L. The equation is then derived from the substitution of the boundary (L + 100) to the grouped equation and collapsing the constants:

$$T(L) = L(C_{shift}) + (L + 100)(C_{alloc}) + C_{init}$$

The allocation of a data array of size L into fixed-size chunks of maximum capacity K (where $K = N^2$) is evaluated by the mathematical induction for the linear time evaluation. Let P(L) be the proposition statement expressing that the allocation of L elements into arrays of fixed size K that needs iterations is directly proportional to L, which is executing in bounded O(L) time. The base case states that If the array length is L = 1, the WHILE loop executes exactly 1 block. The nested loops iterate exactly K times (1 data allocation + K – 1 zero-paddings). Since K is an architectural constant, O(K) = O(1) . P(1) is mathematically true. Assume that P(m) is true meaning processing m elements requires O(m) iterations. Then, the inductive step of P(m + 1) is proceeded to prove.

The algorithm processes the first m elements which evaluates to O(m) time and processes the (m + 1)th element for m + 1 matrix elements. The addition of a single element either fills an existing matrix block adding exactly 1 operation or initiating the creation of another new matrix block adding K operations. It adds exactly K constant operations in the worst-case scenario:

$$T(m + 1) \leq T(m) + K$$

Since K is a non-scaling constant, T(m + 1) remains strictly proportional to m. Therefore, by the principle of mathematical induction, it is true that the allocation process executes in linear time O(L).

The execution time is dominated by the length of the data array L by factoring out the C_{shift} and C_{alloc} operational constants and dropping the non-scaling matrix padding maximum (100) from the evaluation.

$$T(L) = O(L)$$

8) Algorithm 8: Chaotic Key Factor and Matrix Generation

Let c_i be the constant computational time required to execute an atomic arithmetic operation, V_b be the expected iterations to randomly generate a valid base matrix, and V_k be the expected operations to validate the key factor.

Line / Stage	Operation	Cost	Frequency
Stage 1	**Lorenz ODE Chaos Generation**		
2	Initialize variables x, y, z	c_1	1
3-10	FOR i = 1 to Iteration (I) DO	c_2	I
4-6	Compute ODE derivatives (L_x, L_y, L_z)	c_3	I
7-9	Euler update (x, y, z)	c_4	I
11-13	Extract and Round X, Y, Z seeds	c_5	1
Stage 2	**Base Matrix Validation**		
16-22	WHILE IsValidBase == False DO	c_6	V_b
17	Generate random M × M matrix	c_7M^2	V_b
18	Calculate Determinant (Gaussian Elimination)	c_8M^3	V_b
19-21	Modulo validation (3, 7, 21)	c_9	V_b
Stage 3	**Key Factor Validation**		
25-31	WHILE IsValidKeyFactor == False DO	c_{10}	V_k
26-30	IF coprime THEN valid ELSE X + 1	c_{11}	V_k
Stage 4	**Matrix Scaling and Bounding**		
33-34	Scalar Matrix Multiplication	$c_{12}M^2$	1
35	RETURN ValidatedKeyMatrix, X	c_{13}	1

Table 8 – Table method for considering time complexity of Algorithm 8

After the operational blocks are grouped by their scaling factors (I, V_b, V_k, M), the following initial time complexity equation is obtained:

$$T(I,M) = I(c_2 + c_3 + c_4) + V_b(c_6 + c_7M^2 + c_8M^3 + c_9) + V_k(c_{10} + c_{11}) + c_{12}M^2 + (c_1 + c_5 + c_{13})$$

The following structural constants are resolved for simplifying the equation:

a. **The Lorenz Iteration (I):** A fixed cryptographic threshold, such as I = 1000, determines how many repetitions are needed to get a chaotic attractor to leave its initial transitory state. The term $I(c_2 + c_3 + c_4)$ resolves to a strict O(1) startup constant C_{lorenz} since I is an immutable integer that never scales with the file size N.

b. **Matrix Dimensionality (M):** The matrix size is explicitly limited to $M \leq 10$ in the Enochian architecture and the maximum number of the elements that can be put into it is $M^3 \leq 1000$. Since this threshold is mathematically strict, all matrix generation, determinant calculations, and scaling operations are resolved to an O(1) bounded constant C_{matrix}.

c. **Probabilistic Loops (V_b, V_k):** Mathematically, there is a high probability of generating a matrix whose determinant is coprime to 21. Similarly, a maximum of two incremental additions are required to skip multiples of 3 and 7. Consequently, the expected iterations V_b and V_k are small, non-scaling constants.

By transforming and substituting these isolated constants back into the equation:

$$T(N) = C_{lorenz} + C_{matrix} + C_{validations}$$

The mathematical induction is applied for proving the numerical approximation, which is line 3-11, executes in strictly linear time relative to the iteration parameter I for creating the computational bounds of the continuous-time chaos generation. Let P(I) be the proposition that states that iterating the Euler approximation of the Lorenz system for I discrete steps requires exactly I loop executions, evaluating in linear time O(I). The base case states that the loop executes from i = 1 to 1, computing 3 derivatives and updating 3 coordinates using 12 unique arithmetic operations for the exact 1 integration step (I = 1), holding that P(1) is mathematically true. Subsequently, assume that P(m) is true meaning that advancing the state vector by m steps requires exactly m iterations, evaluating in O(m) execution time. Then, the inductive step for P(m + 1) is proved to be true.

The system needs to calculate the trajectory of the first m steps that requires m iterations by the hypothesis in order to compute m + 1 steps. Since the system is a Markovian process, the (m + 1)th state depends on the mth state exclusively. The system executes exactly 1 final iteration to calculate the (m + 1)th coordinate.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since m = (m + 1) – 1, P(m + 1) is considered true. Thus, by the principle of mathematical induction, it is true that the differential equations execute in strictly linear time O(I). However, since I is a cryptographic constant, it falls to constant O(1) relative to the payload size.

The entire algorithm executes as a single, constant-time initialization burst since each and every parameter used in the generation (I, M, coprimes) consists of fixed mathematical bounds that are entirely independent of the length of the plaintext payload (N).

$$T(N) = O(1)$$

9) Algorithm 9: Chaotic Seed Extraction and Assignment

Let c_i be the constant computational time required to execute an atomic mathematical rounding or memory assignment operation.

Line	Operation	Cost	Frequency
2	Integer (Y) ← RoundToInt(y_{final})	c_1	1
3	Integer (Z) ← RoundToInt(z_{final})	c_2	1
5	GlobalSession.SBoxSeed ← Integer(Y)	c_3	1
6	GlobalSession.HashBase ← Integer(Z)	c_4	1
7	RETURN Integer(Y), Integer(Z)	c_5	1

Table 9 – Table method for considering time complexity of Algorithm 9

From table 9, the following initial time complexity equation is obtained:

$$T(N) = c_1 + c_2 + c_3 + c_4 + c_5$$

The frequency of execution for all line is exactly one because a scalar floating-point coordinate is processed in every single operation in the algorithm and assigned it to a global variable. The payload length N relies on no array traversals, no loops, and no variable transformation. Let $C_{extract}$ be the grouped sum of these unique constants:

$$T(N) = C_{extract}$$

The mathematical induction is applied for proving the dimensional independence in algorithm without iterative structures or recursive structures. Let P(N) be the proposition states the algorithm executes in a bounded constant time K and is strictly independent of the payload size N.

The base case states that the algorithm rounds 2 scalar variables to nearest integers and assigns them to 2 global pointers if the plaintext payload includes only one character (N = 1). It executes in K operations exactly and hence P(1) is considered to be true. Then the following inductive hypothesis P(m) is assumed to be true. That is, the extraction executes in K operations exactly for a payload of m characters. Then, the inductive step of a payload m + 1 characters is continued to prove.

Since the chaos seeds generation algorithm only receives the singular y_{final} and z_{final} floating-point lastly iterated output from the ODE generator, the function never passes the addition of the (m + 1)th plaintext character, and the system only rounds 2 scalar variables and assigns 2 pointers.

$$T(m+1) = K$$

Since $T(m+1) = T(m) = K$, the condition is true. Thus, by the principle of mathematical induction, it is proved that the execution time of the algorithm is constant time O(1) strictly.

Since the total number of operations falls to a singular non-scaling constant $C_{extract}$, the absolute constant time complexity is achieved.

$$T(N) = O(1)$$

10) Algorithm 10: Chaotic S-Box Execution

Let c_i be the constant computational time needed to execute the basic atomic operation such as array allocation, PRNG generation, and array indexing, B be the total number of matrix blocks in the ValueList, and M be the dimensional size of the matrices.

Line	Operation	Cost	Frequency
2	Initialize SBox [1 to 21]	c_1	1
3	InitializeRandomGenerator(Y)	c_2	1
4	ScrambleArray(SBox, PRNG)	c_3	1
5	Initialize SubstitutedValueList	c_4	1
7-20	FOR EACH Matrix IN ValueList DO	c_5	B
8	Create SubstitutedMatrix[Rows, Cols]	c_6	B
9-18	Nested FOR loops (r and c)	c_7	B . M^2
11-16	Original Value extraction & IF/ELSE	c_8	B . M^2
19	SubstitutedValueList.Append	c_9	B
21	RETURN SubstitutedValueList	c_{10}	1

Table 10 – Table method for considering time complexity of Algorithm 10

The following initial time complexity equation is obtained from table 10:

$$T(B, M) = c_1 + c_2 + c_3 + c_4 + B(c_5 + c_6 + c_9) + (B.M^2)(c_7 + c_8) + c_{10}$$

From the above equation, there are two things that needs to be solved. The first one is the addressing of the Enochian constants. An array of exactly 21 elements is carefully processed throughout the S-Box initialization and scrambling, which is lines 2-4. Scrambling the Enochian alphabet requires a fixed 21 operations since its size never expands in relation to the file size. Consequently, $c_1 + c_2 + c_3$ falls into C_{sbox}, a strict O(1) scalar initialization constant.

The second resolution is the matrix dimensionality. The total number of matrix cell elements L included across all payload matrices is represented as the term B . M^2:

$$L = B.M^2$$

Thus, the equation can be rewritten in term of the total payload size L as:

$$T(L) = C_{sbox} + c_4 + B(c_5 + c_6 + c_9) + L(c_7 + c_8) + c_{10}$$

Since the dimension of the matrix M is limited its minimum to $M \geq 2$ strictly, the number of blocks B will never exceed $\frac{L}{4}$, and thus B scales strictly linearly with L. By grouping the coefficients:

$$T(L) = L(C_{substitution}) + C_{init}$$

The mathematical induction is used to prove that mapping an N × N matrix element-by-element requires iterations strictly proportional to the number of elements K inside the matrix (where $K = N^2$) in order to strictly create the boundaries of the substitution process. Let P(K) be the proposition that states that it exactly takes K inner-loop executions to substitute K total items by iterating across a two-dimensional grid in limited O(K) time. The base case states that the nested loops execute exactly 1 time if the matrix contains K = 1 element (a 1 × 1 grid). It also performs 1 lookup and evaluates 1 conditional statement. Hence, it implies that P(1) mathematically holds true. The inductive hypothesis assumes P(m) is true meaning that substituting a matrix with m entries takes O(m) time and executes precisely m times. The inductive step for evaluating a matrix containing m + 1 elements is continued to prove.

The algorithm substitutes the first m element (requiring m executions by our hypothesis) and performs 1 additional lookup for the (m + 1)th element.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since m = (m + 1) – 1, The statement P(m + 1) is logically held. Thus, by the principle of mathematical induction, it is proved that the total number of inner-loop executions is mathematically fixed to the total element count L when it is applied across B blocks.

The execution time is strictly dominated by the entire payload mass L after factoring out the $C_{substitution}$ operational constant and dropping the O(1) S-Box generation and return constants ($C_{sbox} + C_{init}$) from the equation.

$$T(L) = O(L)$$

11) Algorithm 11: Hill Cipher Matrix Multiplication

Let c_i be the constant computational time required for a fundamental atomic operation, N be the total character length of the plaintext payload, B be the total number of matrix blocks where $B = \frac{N}{M^2}$, and M be the dimensional size of the matrices.

Line	Operation	Cost	Frequency
1	InitializeEncryptedValueList	c_1	1
2-14	FOR EACH SubstituteMatrix IN …	c_2	B
4	MultiplyMatrices(KeyMatrix, Sub)	c_3	$B \cdot M^3$
5-6	Rows, Cols ← GetCount(…)	c_4	B
8-12	Nested Modulo Loops (r and c)	c_5	$B \cdot M^2$
10	EncryptedMatrix[r, c] mod 21	c_6	$B \cdot M^2$
13	EncryptedValueList.Append(…)	c_7	B
15	RETURN EncryptedValueList	c_8	1

Table 11 – Table method for considering time complexity of Algorithm 11

The following initial time complexity equation is obtained from table 11:

$$T(B, M) = c_1 + B(c_2 + c_4 + c_7) + B(c_3 M^3) + B.M^2(c_5 + c_6) + c_8$$

According to the standard matrix multiplication algorithm, the exact M^3 operations are needed to multiply two M × M matrices. Similarly, M^2 operations are required to apply the modulo limit by iterating across the grid. Combining the block-level costs results in:

$$C_{block} = c_3 M^3 + M^2(c_5 + c_6) + (c_2 + c_4 + c_7)$$

The matrix dimension is strictly limited by an upper bound of 10 in the created Enochian architecture.

$$M \leq 10 \Rightarrow M^3 \leq 1000 \; and \; M^2 \leq 100$$

The whole block calculation reduces to an isolated O(1) execution constraint since the mathematical ceiling of 1000 operations per block is an immutable architecture constant that never grows with the payload N. Substituting C_{block} back into the equation:

$$T(N) = B(C_{block}) + (c_1 + c_8)$$

The mathematical induction is then used for proving that evaluating B total localized blocks scales linearly with the total payload length N. Let P(N) be a proposition states that B executions are required to partition a payload of length N into fixed blocks of maximum capacity M^2 in limited O(N) time. The base case states that the element number fits entirely within the minimum lower bound of a single 2 × 2 block if the payload data contains N = 1 character exactly. Then, the multiplication loop executes exactly 1 block (B = 1) and hence P(1) is mathematically true. Assume the inductive hypothesis of P(k) is true. The direct linear time computation is scaled for processing a payload of length k that executes B_k block iterations. The inductive step for the evaluation of a payload of $k + M^2$ elements is continued to prove. (Note that $k + M^2$ means adding one full block's worth of data)

The system multiplies the remaining M^2 elements using exactly 1 more block iteration after processing the first k elements that requires B_k blocks.

$$B_{(k+M^2)} = B_k + 1$$

It is obvious that total execution time grows linearly since the addition of a strictly limited data block causes exactly one extra O(1) constant-time execution. Hence, by the principle of mathematical induction, it is proved the mathematical relationship $B = \left\lceil \frac{N}{M^2} \right\rceil$ ensures that the growth of B is directly proportional to the growth of N.

The total payload N dominates the execution time linearly by factoring out the strictly bounded O(1) operations that are necessary for processing the inner M × M matrix computations.

$$T(N) = O(N)$$

12) Algorithm 12: Rolling Hash Tagging and Shuffle

Let c_i be the constant computational time required to execute a fundamental atomic operation, N be the total number of matrix cards within the EncryptedValueList. It is important to note that Fisher-Yates algorithm is universally proved algorithm that executes in linear operations O(N) exactly by swapping each element exactly once.

Line	Operation	Cost	Frequency
1	Initialize Deck ← Empty Array	c_1	1
2	PreviousHash ← 0	c_2	1
3-11	FOR i = 1 to length(List) DO	c_3	N
4	Extract CurrentMatrix	c_4	N
6	Calculate CurrentHash	c_5	N
8	CreateCard(…)	c_6	N
9	Deck.Append(NewCard)	c_7	N
10	PreviousHash ← CurrentHash	c_8	N
12	FisherYatesShuffle(Deck)	$c_{shuffle}$	N
13	RETURN ShuffledDeck	c_9	1

Table 12 – Table method for considering time complexity of Algorithm 12

The following initial time complexity equation is yielded from table 12:

$$T(N) = c_1 + c_2 + c_3N + c_4N + c_5N + c_6N + c_7N + c_8N + C_{shuffle}N + c_9$$

By grouping the scaling loop operations and the Fisher-Yates execution constant:

$$T(N) = N\left(c_3 + c_4 + c_5 + c_6 + c_7 + c_8 + C_{shuffle}\right) + (c_1 + c_2 + c_3)$$

The mathematical induction is applied to prove that sequentially hashing and appending N cards executes in strictly linear time for strictly defining the bounds of the tagging mechanism. Let P(N) be the proposition statement stating that a rolling hash calculation and object payload construction needs N iterations exactly in iterating through an array of length N. The base case expresses that the loop iterates from i = 1 to 1 if the matrix card list includes exactly N = 1 matrix block. The loop processes 1 iteration exactly and that iteration is performed with 1 mathematical hash calculation, 1 object instantiation, and 1 array append together. This makes base case P(1)

mathematically true. The inductive hypothesis assumes P(m) is true. It means there are m loop executions exactly needed for processing m matrix cards. Then, the inductive step of a matrix list of length m + 1 is proved subsequently.

In order to compute the hash of the (m + 1)th element and append it to the deck, the algorithm must process the first m elements, which needs m iterations according to the previous inductive hypothesis, and perform an additional 1 evaluation loop.

$$T(m+1) = T(m) + 1 = m + 1$$

Since $m = (m+1) - 1$, P(m + 1) is held logically. Thus, by the principle of mathematical induction, it is true that the hashing loop evaluates in linear operation O(N) strictly.

The payload block count N absolutely dominates the execution time when the operational loop coefficients, linear Fisher-Yates constant, and O(1) scalar initialization constants are removed from the Table Method equation.

$$T(N) = O(N)$$

13) Algorithm 13: Subset-Sum Signature Generation

Let c_i be the constant computational time needed for executing a fundamental atomic operation and N be the architectural length of the sender's private key array PR_{tx}.

Line	Operation	Cost	Frequency
1	Initialize V_{vig} binary array	c_1	1
2	TargetRemainder ← TargetHeader	c_2	1
4-11	FOR i = length(PR_{tx}) – 1 DOWNTO 0 DO	c_3	N
5	IF PR_{tx}[i] <= TargetRemainder	c_4	N
6-7	THEN V_{sig}[i] ← 1, Subtract remainder	c_5	N (worst-case)
9	ELSE V_{sig}[i] ← 0	c_6	N (worst-case)
13-15	IF TargetRemainder != 0 THEN HALT	c_7	1
16	RETURN V_{sig}	c_8	1

Table 13 – Table method for considering time complexity of Algorithm 13

The initial time complexity equation is derived from the equation 13:

$$T(N) = c_1 + c_2 + c_3N + c_4N + max(c_5N, c_6N) + c_7 + c_8$$

Either the THEN assignment and subtraction (cost c_5) or the ELSE zero assignment (cost c_6) are performed by the conditional blocks (Lines 6-11). It is assumed that the most computationally expensive branch is performed in each iteration for asymptotic worst-case limits. Let C_{branch} = max(c_5, c_6). By combining the non-scaling constants with the scaling loop operations:

$$T(N) = N(c_3 + c_4 + C_{branch}) + (c_1 + c_2 + c_7 + c_8)$$

The mathematical induction is used for proving that evaluating the mathematical trapdoor executes in linear time exactly with respect to the private key length N in order to strictly verify the boundaries of the subset-sum evaluation method. Let(P) be the proposition stating that using a single-pass greedy algorithm, the trapdoor can be resolved in limited linear time with N iterations exactly against a private key array of size N. The base case states that the reverse FOR loop iterates from i = 0 down to 0 if the private key contains exactly N = 1 element. It performs a 1 conditional check and updates the target remainder in an exact 1 iteration and thus mathematically, P(1) is true. The inductive hypothesis expresses that evaluating an array of size m requires exactly m iterations, scaling directly in O(m) execution time assuming that P(m) is true. Then , the inductive step for evaluating a key array of size m + 1 is continued to prove.

According to the inductive hypothesis, the algorithm must process the first m entries in the array, which calls for m iterations. For the (m + 1)th element, it must execute an extra 1 evaluation loop.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since m = (m + 1) – 1, P(m + 1) logically holds. Thus, by the principle of mathematical induction, it is proved that the trapdoor resolution loop evaluates in strictly linear O(N) operations.

The scaling term N rigorously dominates the execution time by factoring out the constant loop coefficients ($c_3 + c_4 + C_{branch}$) and eliminating the O(1) initialization and return constants from the Table Method equation.

$$T(N) = O(N)$$

14) Algorithm 14: Payload Serialization

Let c_i be the constant computational time required to execute an atomic operation such as allocating a container or passing a pointer, c_{ser} be the time to serialize a single byte/element, c_{net} be the time to buffer a single byte for network transmission, N be the total combined byte or element size of the TransmittablePayload.

Line	Operation	Cost	Frequency
1	Initialize TransmittablePayload	c_1	1
3-6	Append (Header, Signature, Deck)	c_2	3
7	Serialize (TransmittablePayload)	c_{ser}	N
8	TransmitToNetwork(SerializedData)	c_{net}	N
9	RETURN Status_Sucess	c_3	1

Table 14 – Table method for considering time complexity of Algorithm 14

The initial time complexity equation is derived from table 14:

$$T(N) = c_1 + 3c_2 + c_{ser}N + c_{net}N + c_3$$

The algebraic equation for the algorithm's execution time is established by grouping the linear streaming operations and non-scaling initialization constants together:

$$T(N) = N(c_{ser} + c_{net}) + (c_1 + 3c_2 + c_3)$$

The mathematical induction is used to prove that converting and streaming a data structure of total size N occurs in exactly linear time, therefore rigorously establishing the boundaries of the serialization and transmission process. Let P(N) be the proposition stating that iterating through a combined payload structure to serialize and transmit exactly N elements requires N iterations executing in bounded linear time. The base case states that the serialization process reads exactly 1 memory block, converts it, and the network buffer transmits exactly 1 byte if the total payload size is exactly N = 1 element or byte and hence, P(1) is holds true mathematically. The following inductive hypothesis assumes that streaming a payload of size m requires exactly m sequential conversions and transmission, scaling directly in O(m) execution time which P(m) is true. Then, the inductive step for a payload of size m + 1 is proceeded to prove for evaluation.

According to the inductive hypothesis, the algorithm must process the first m bytes of the payload stream, which operates for exactly m operations. For the (m + 1)th byte, it must do an extra 1 conversion and buffer push.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since m = (m + 1) – 1, P(m + 1) is logically true. Hence, by the principle of mathematical induction, it is proved that the serialization and network transmission sequence evaluates in strictly linear O(N) operations.

The payload size N strictly dominates the entire execution time by eliminating the isolated O(1) pointer assignment constants and factoring out c_{ser} and c_{net} operational coefficients from the Table Method equation.

$$T(N) = O(N)$$

15) Algorithm 15: Digital Signature Verification

Let c_i be the constant computational time needed for executing an atomic arithmetic or assignment operation, E be the computational time required for resolving the Extended Euclidean algorithm for the modular inverse, N be the architectural length of the binary signature vector and the sender's public key array.

Line	Operation	Cost	Frequency
1	Initialize Checksum ← 0	c_1	1
3-5	FOR i = 0 to length(V_{sig}) – 1 DO	c_2	N
4	Checksum ← Checksum + (V_{sig}[i] * PU_{tx}[i])	c_3	N
7	Inv_N ← CalculateModularInverse(…)	E	1
8	VerificationResult ← (Checksum * Inv_N) mod M_{tx}	c_4	1
10-14	IF / ELSE Integrity Check and RETURN	c_5	1

Table 15 – Table method for considering time complexity of Algorithm 15

The following initial time complexity equation is derived from the table 15:

$$T(N) = c_1 + c_2N + c_3N + E + c_4 + c_5$$

The Extended Euclidean Algorithm, which evaluates in O(log(min(N_{tx}, M_{tx}))) time, is used by the CalculateModularInverse function. The bit-lengths of the public multiplier (N_{tx}) and

modulus (M_{tx}) do not scale with the vector length N as they are fixed scalar variables created during the asymmetric key handshake. Consequently, an isolated O(1) cryptographic constant C_{inv} is the result of the full modular inverse computation.

By substituting C_{inv} and grouping the linear operations:

$$T(N) = N(c_2 + c_3) + (c_1 + C_{inv} + c_4 + c_5)$$

To explicitly prove the closed-form execution time of the vector dot product loop, which are lines 4 and 6) in relation to the length of the signature array N, the mathematical induction is used. Let T(k) be the total number of operations needed to calculate the dot product for k items. The following recurrence definition operates how the algorithm's loop functions:

$$T(1) = c_2 + c_3$$
$$T(k) = T(k-1) + (c_2 + c_3)(fork > 1)$$

From the definition, the following proposition P(N) can be stated: the closed-form execution time for the checksum loop is mathematically defined as $T(N) = N(c_2 + c_3)$. The base case states that the closed-form equation evaluates to $1(c_2 + c_3) = c_2 + c_3$ if the signature vector contains exactly N = 1 element. This perfectly matches the recurrence relation's original definition and hence P(1) is true. By inductive hypothesis, assuming P(m) is also true for some arbitrary positive integer $m \geq 1$:

$$T(m) = m(c_2 + c_3)$$

Then, the inductive step of P(m + 1) is proceeded to prove. According to the recurrence's basic definition, evaluating m+1 elements necessitates adding the constant cost of the single additional iteration to the total time of m elements:

$$T(m+1) = T(m) + (c_2 + c_3)$$

Substituting T(m) with the inductive hypothesis:

$$T(m+1) = m(c_2 + c_3) + (c_2 + c_3)$$

Factoring out the constant coefficient $(c_2 + c_3)$ produces:

$$T(m+1) = (m+1)(c_2 + c_3)$$

The output equation exactly reflects the original proposition when it is evaluated at $N = m + 1$. Hence, by the principle of mathematical induction, it is true that the checksum loop is scaled in linear operation strictly.

The vector length N absolutely dominates the execution time when the operational constants are dropped and the non-scaling cryptographic modular inverse C_{inv} is removed from the Table Method equation.

$$T(N) = O(N)$$

16) Algorithm 16: Subset-Sum Key Decapsulation

Let c_i represent the constant computational time required to execute a fundamental atomic operation, E represent the computational time needed for solving the Extended Euclidean Algorithm for the modular inverse, and N be the architectural length of the receiver's private key array. The procedural execution workflow of the subset-sum decapsulation is evaluated using the table method.

Line	Operation	Cost	Frequency
1	$Inv_N \leftarrow$ CalculateModularInverse(…)	E	1
2	TargetSum $\leftarrow$ (H_{enc} * Inv_N) mod M_{rx}	c_1	1
3	Initialize $V_{session}$ binary array	c_2	1
4	CurrentSum $\leftarrow$ TargetSum	c_3	1
6-13	FOR i = length(PR_{rx}) – 1 DOWNTO 0 DO	c_4	N
7	IF PR_{rx}[i] <= CurrentSum	c_5	N
8-9	THEN $V_{session}$[i] $\leftarrow$ 1, Subtract sum	c_6	N (worst-case)
11	ELSE $V_{session}$[i] $\leftarrow$ 0	c_7	N (worst-case)
15-17	IF CurrentSum != 0 THEN HALT	c_8	1
18	RETURN $V_{session}$	c_9	1

Table 16 – Table method for considering time complexity of Algorithm 16

The initial time complexity equation is derived from the table 16:

$$T(N) = E + c_1 + c_2 + c_3 + c_4N + c_5N + max(c_6N, c_7N) + c_8 + c_9$$

The Extended Euclidean Algorithm is used by the CalculateModularInverse function. The bit-lengths of the public multiplier (N_{rx}) and modulus (M_{rx}) do not scale proportionately with the vector length N as they are tightly bounded cryptographic scalars created during the session setup. Consequently, an isolated, non-scaling O(1) cryptographic constant C_{inv} results from the full modular inverse computation collapsing. To get the worst-case bound fort the conditional block, it is assumed that the most computationally costly branch $C_{branch} = \max(c_6, c_7)$ is performed on each iteration.

By grouping the linear scaling operations and the isolated constants:

$$T(N) = N(c_4 + c_5 + C_{branch}) + (C_{inv} + c_1 + c_2 + c_3 + c_8 + c_9)$$

Then the mathematical induction is applied to prove that the mathematical trapdoor execution is evaluated in strictly linear time relative to the private key length N for creating the asymptotic bounds of the decapsulation process. Let P(N) be the proposition stating that using a single-pass greedy algorithm, the decapsulation trapdoor against a private key array of size N can be resolved in bounded linear time with exactly N iterations. The base case states that the reverse FOR loop iterates from i = 0 down to 0 if the private key contains exactly N = 1 element. It performs exactly 1 iteration, 1 conditional check and conditionally updating the target remainder. Hence, P(1) is true mathematically. Then inductive hypothesis assumes that evaluating an array of size m requires exactly m iterations, scaling directly in O(m) execution time which means P(m) is true. Then the inductive step for the key array of size m + 1 is proceeded to prove.

According to the inductive hypothesis, the algorithm must evaluate the array's first m elements, which calls for m iterations. For the (m + 1)th element, it must execute an extra 1 evaluation loop.

$$T(m + 1) = T(m) + 1 = m + 1$$

Since m = (m + 1) – 1, P(m + 1) is logically true. Therefore, by the principle of mathematical induction, it is true that the trapdoor resolution loop evaluates in strictly linear operations O(N).

The scaling term N strictly dominates the execution time after factoring out the constant loop coefficients and eliminating the O(1) initialization and modular inverse constants from the Table Method equation.

$$T(N) = O(N)$$

17) Algorithm 17: Payload Sorting and Key Regeneration

Let c_i represents the constant computational time for structural operations, and N represents the total number of matrix cards in the intercepted payload deck. The independent recurrence functions $T_{sort}(N)$, and $T_{query}(N)$ are isolated because the sorting and searching functions dominate the execution time.

Line	Operation	Cost	Frequency
2-8	Extract ExpectedHashSequence	c_1	N
11	Introsort(Deck, by: HashTag)	$T_{sort}(N)$	1
13-17	Initialize ReconstructedPayload	c_2	N
14	FOR EACH ExpectedHash IN Sequence DO	c_3	N
14-16	TargetCard ← BinarySearch(…)	$T_{query}(N)$	N
19-24	ReconstructedPayload.Append(…)	c_4	1
25	RETURN ReconstructedPayload	c_5	1

Table 17 – Table method for considering time complexity of Algorithm 17

The initial time complexity equation is derived from the table 17:

$$T(N) = T_{sort}(N) + N.T_{query}(N) + N(c_3 + c_4) + (c_1 + c_2 + c_5)$$

The payload partition is initiated by Introsort using QuickSort. A pivot is selected and the array is divided into two sub-arrays, recursively calling itself on halves of the dataset. It takes precisely linear O(N) time to partition the items.

The sorting operation is defined in the following recurrence relation:

$$T_{sort}(N) = 2T\left(\frac{N}{2}\right) + cN$$

The Master Theorem is applied in the standard form $T(N) = aT\left(\frac{N}{b}\right) + f(N)$. The array is divided into 2 sub-problems and hence branching factor (a) is defined as 2. The scale factor (b) is 2 because each sub-problem is half the size of the original. The work function (f(N)) is $\Theta(N^1)$ because the partition cost is linear. The critical exponent c_{crit} is calculated:

$$c_{crit} = log_b\, a = log_2\, 2 = 1$$

Since $f(N) = \Theta(N^1)$ matches the critical bound $N^{c_{crit}} = N^1$, the recurrence relation perfectly satisfies with the second case of the Master Theorem. The final asymptotic bound is derived by multiplying the work function by the logarithmic recursion depth:

$$T_{sort}(N) = \Theta(NlogN)$$

Introsort dynamically pivots to Heap Sort if the recursion tree depth is beyond 2 $\log_2$ N. The absolute worst-case time complexity remains strictly concealed at O(N log N) since Heap Sort is mathematically bounded at O(N log N).

Then, in the part of matrix card retrieval, the sorted deck is queried using the Binary Search by the algorithm. In each recursive step, Binary Search divides the search space in half without splitting into several smaller sub-problems. The Master Theorem is applied for defining a single search query in the following recurrence relation:

$$T_{query}(N) = 1T\left(\frac{N}{2}\right) + c$$

Since only one half of the array is explored, the branching factor (a) is 1. The search space is halved and thus the scale factor (b) is 2. The work function f(N) is $\Theta(1) = \Theta(N^0)$ meaning it takes the constant time to check the midpoint. The critical exponent c_{crit} is calculated:

$$c_{crit} = log_b\, a = log_2\, 1 = 0$$

The critical bound $N^{c_{crit}} = N^0$ is matched with the $f(N) = \Theta(N^0)$ and thus the relation is satisfied with the second case of the Master Theorem.

$$T_{query}(N) = \Theta(logN)$$

Since there are N individual matrix cards to be retrieved, the total retrieval time is:

$$Total\ Retrieval = N.T_{query}(N) = N.O(log\ N) = O(N\ log\ N)$$

Substituting the strictly proven Master Theorem bounds back into the initial Table Method equation:

$$T(N) = O(NlogN) + O(NlogN) + N(c_3 + c_4) + C_{init}$$

The linear and constant variables dominate the equation since O(N log N) is the highest-order scaling term.

$$T(N) = O(Nlog\ N)$$

18) Algorithm 18: Decryption and Reverse Substitution

Let c_i be the constant computational time needed for executing a fundamental atomic operation, B be the total number of intercepted matrix blocks where $B = \frac{N}{M^2}$, M be the dimensional size of the matrices.

Line	Operation	Cost	Frequency
10	Initialize DecryptedValueList	c_1	1
12-36	FOR EACH EncryptedMatrix IN Payload	c_2	B
2-5	InvKeyMatrix ← ComputeInverse(KeyMatrix)	$c_{inv}M^3$	1
15	DecryptedMatrix ← Multiply(…)	c_3M^3	B
20-34	Nested Modulo & S-Box loops (r and c)	c_4	B . M^2
22	Value ← DecryptedMatrix[r, c] mod 21	c_5	B . M^2
29	OriginalVal ← InverseSBox(Value)	c_6	B . M^2
35	DecryptedValueList.Append(…)	c_7	B
37	RETURN DecryptedValueList	c_8	1

Table 18 – Table method for considering time complexity of Algorithm 18

The following initial time complexity equation is derived from table 18.

$$T(B, M) = c_1 + B(c_2 + c_7) + B(c_{inv}M^3 + c_3M^3) + B.M^2(c_4 + c_5 + c_6) + c_8$$

Cubic M^3 operations are needed to calculate the inverse of a M × M matrix and multiply it. M^2 operations are needed to extract the values and run them through the inverse S-Box. The block-level decryption charges are grouped as follows:

$$C_{block} = (c_{inv} + c_3)M^3 + (c_4 + c_5 + c_6)M^2 + (c_2 + c_7)$$

In Enochian architecture, the matrix dimension M is restricted by a maximum boundary of 10.

$$M \leq 10 \implies M^3 \leq 1000 \; and \; M^2 \leq 100$$

The whole matrix inversion and replacement block resolves to an isolated O(1) constant execution constraint since the structural maximum of 1000 operations per block is an immutable constant that never grows with the file size N.

Substituting C_{block} back into the grouped equation:

$$T(N) = B(C_{block}) + (c_1 + c_8)$$

The mathematical induction is applied for explicitly proving that decrypting B total localized matrix blocks grows strictly linearly with the full ciphertext length N. Let P(N) be the proposition stating that B sequential block executions are required for decrypting a ciphertext payload of total length N partitioned into blocks of maximum capacity of M^2 which is evaluating in bounded O(N) time. The base case states that it takes a single matrix block if the ciphertext is exactly N = 1 character long. 1 block inversion and multiplication is exactly executed by the main loop B = 1. Thus, P(1) is mathematically true. Then, the inductive hypothesis assumes that B_k block inversions are performed while processing a ciphertext of length k, scaling directly in O(k) time, which means P(m) is true. The following inductive step for a ciphertext of $k + M^2$ elements which is representing the addition of one completely full block is proceeded to prove.

The algorithm requires precisely one more O(1) block iteration to invert and replace the remaining M^2 items after decrypting the first k elements which requires B_k block inversions according to the inductive hypothesis.

$$B_{(k+M^2)} = B_k + 1$$

The total time grows perfectly linearly since the addition of a dimensionally limited ciphertext block yields exactly one more O(1) constant-time operation. Hence, by the principle of mathematical induction, it is proved that the mathematical mapping $B = \left\lceil \frac{N}{M^2} \right\rceil$ strictly ensures that the growth of B is directly proportional to the expansion of N.

The total ciphertext payload mass N mathematically dominates the execution runtime by eliminating the scalar initialization parameters and factoring out the strictly limited O(1) cubic matrix operations.

$$T(N) = O(N)$$

19) Algorithm 19: Remapping and Plaintext Assembly

Let c_i be the constant computational time needed for the execution of a fundamental atomic operation such as array extraction, arithmetic subtraction, and memory appending, N be the total character length of the fully decrypted payload array.

Line	Operation	Cost	Frequency
1,12,21	Initialize Plaintext ← Empty String	c_1	3
3-10	FOR EACH Value IN DecryptedValueList DO	c_2	N
23-24	ShiftedIndex ← (Value – C_1) mod 26	c_3	N
17	TargetChar ← AlphabetMap[ShiftedIndex]	c_4	N
7,18,25	Plaintext.Append(TargetChar)	c_5	N
30	RETURN Plaintext	c_6	1

Table 19 – Table method for considering time complexity of Algorithm 19

The initial time complexity equation is derived from the table 19:

$$T(N) = 3c_1 + c_2N + c_3N + c_4N + c_5N + c_6$$

The core algebraic equation is obtained by strictly grouping the scaling iterative operations separately from the non-scaling instantiation and return constants:

$$T(N) = N(c_2 + c_3 + c_4 + c_5) + (3c_1 + c_6)$$

The closed-form execution time of the assembly loop against the data array length N is proved by using the mathematical induction. Let T(k) be the required total operations for mapping and appending k characters. The loop operates under the following strictly defined recurrence relation:

$$T(1) = c_2 + c_3 + c_4 + c_5$$

$$T(k) = T(k-1) + (c_2 + c_3 + c_4 + c_5) \quad for\ k > 1$$

After defining the T(K), the proposition P(N) can be started stating: the closed-form execution time for the plaintext assembly loop is mathematically defined as $T(N) = N(c_2 + c_3 + c_4 + c_5)$. The base case states that the closed-form equation evaluates to $1(c_2 + c_3 + c_4 + c_5)$ if the decrypted payload contains exactly N = 1. This perfectly matches with the recurrence relation's fundamental

definition and hence P(1) is mathematically true. Then, the inductive hypothesis assumes that for some arbitrary positive integer m ≥ 1, the proposition P(m) is true:

$$T(m) = m(c_2 + c_3 + c_4 + c_5)$$

The inductive step for P(m + 1) is proved for its correctness. Processing m + 1 items needs adding the constant operational cost of the one additional iteration to the total time spent processing m values according to the recurrence's basic definition:

$$T(m + 1) = T(m) + (c_2 + c_3 + c_4 + c_5)$$

Substituting T(m) with the previously established inductive hypothesis:

$$T(m + 1) = m(c_2 + c_3 + c_4 + c_5) + (c_2 + c_3 + c_4 + c_5)$$

The final expanded form is obtained by factoring out the constant coefficients:

$$T(m + 1) = (m + 1)(c_2 + c_3 + c_4 + c_5)$$

This algebraic equation, which is now evaluated at N = m + 1, exactly reflects the original proposition. Thus, by the principle of mathematical induction, it is proved that the assembly loop strictly scales in precisely O(N) operations since the inductive step and the base case are both mathematically satisfied.

The entire plaintext mass N strictly bounds the overall execution runtime by factoring out the uniform operational loop constants and dropping the isolated O(1) startup bounds from the Table Method equation.

$$T(N) = O(N)$$

2. Space Complexity Calculation of Algorithms

1) Algorithm 1: Receiver Knapsack Key Generation

Let N be the architectural length of the generated cryptographic sequence. The algorithm 1 evaluates the dynamic memory allocation during the execution environment. Here, the resources for the algorithmic instructions are excluded and only data structures constructed at execution are counted.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
PU_{rx} (Public Key)	Integer Array	1 × N	c . N
PR_{rx} (Private Key)	Integer Array	1 × N	c . N
TotalSum	Integer (Scalar)	1 × 1	c
M_{rx} / N_{rx}	Integer (Scalar)	1 × 2	2c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 1 – Table method for considering space complexity of Algorithm 1

The total auxiliary space S(N) is the sum of the allocated data structures:

$$S(N) = c.N + c.N + c + 2c + c$$

$$S(N) = 2cN + 4c$$

The maximum memory output created in the call stack at any given point during execution is evaluated for establishing the closed-form space complexity. Let $S_{prop}(N)$ be the proposition stating that total memory allocated by the algorithm for generating a sequence of length N scaling strictly linearly in proportion to N. The scalar primitives in this algorithm are the variables named TotalSum, modulus and multiplier constants, and the iterator i. These values are overwritten in place within the same memory registers while the loop iteration is in progress. Regardless of the sequence length N, the overall memory output is limited to the constant 4c since additional memory addresses are not dynamically assigned every iteration.

The contiguous memory allocation is required for exact N elements by both the PU_{rx} and PR_{rx} array. Since each element of array takes a constant word size c, the array strictly consumes 2cN memory space. The non-scaling scalar constant 4c and the operational word coefficient are dropped by isolating the highest-order scaling term from the closed-form equation.

$$S(N) = O(N)$$

2) Algorithm 2: Sender Knapsack Key Generation

Let N be the architectural length of the generated public key array. The dynamic memory allocation is evaluated during the runtime execution. The cryptographic multiplier and modulus are represented as variables and evaluated as isolated scalars.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
PU_{tx} (Public Key)	Integer Array	1 × N	c . N
PR_{tx} (Private Key)	Integer Array	1 × N	c . N
TotalSum	Integer (Scalar)	1 × 1	c
M_{tx} / N_{tx}	Integer (Scalar)	1 × 2	2c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 2 – Table method for considering space complexity of Algorithm 2

The sum of all dynamically allocated data structures during the transformation loop is the total auxiliary space S(N):

$$S(N) = 2cN + 4c$$

The maximum memory output created in the call stack at any given point during execution is evaluated for strictly creating the closed-form space complexity. Let $S_{prop}(N)$ be the total memory allocated by the algorithm to produce a public sequence of length N scales strictly linearly in proportion to N. The scalar primitive variables of the algorithm are TotalSum, N_{tx}, M_{tx}, and the iterator i. The values are access and overwritten in place within the same memory registers while the iterative multiplication loop is in progress. The total scalar memory output is bounded to the constant 4c, no matter the length of sequence N, because new memory addresses are not allocated dynamically per iteration.

The array allocations for the public and private keys consumes linear space. The non-scaling scalar constant 4c and the operational word coefficient are dropped by separating the highest-order scaling term from the closed-form equation.

$$S(N) = O(N)$$

3) Algorithm 3: Chaotic Parameter Initialization

The runtime execution environment's dynamic memory allocation is evaluated. System parameters and differential state variables are evaluated as separate numerical scalars.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
$V_{session}$ (Session Vector)	Binary Array	1 × 3	3c
C1, C2 (Shifts)	Integer (Scalar)	1 × 2	2c
L_x, L_y, L_z (Coordinates)	Float (Scalar)	1 × 3	3c
VectorSum	Integer (Scalar)	1 × 1	c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 3 – Table method for considering space complexity of Algorithm 3

The sum of the memory blocks that are dynamically allocated throughout the integration loop is represented by the total auxiliary space S:

$$S = 3c + 2c + 3c + c + c$$

$$S = 10c$$

The highest memory usage allocated in the call stack is evaluated in order to strictly create the closed-form space complexity. Let S_{prop} be the proposition stating that total memory allocated for generating the ephemeral session parameters remains strictly constant. The fundamental scalar primitives (C1, C2, L_x, L_y, L_z, VectorSum, and i) are initialized and stored in fixed memory registers. The $V_{session}$ data structure is an array but its architectural dimensionality is strictly capped at 3 binary elements. The scalar memory usage is remained bound up to the constant 10c since there is no new memory allocation in each integration step dynamically.

The operational word coefficient c and the static integer multiplier are eliminated from the closed-form equation to establish absolute constant time.

$$S = O(1)$$

4) Algorithm 4: Session Vector Encapsulation

The runtime execution environment's dynamic memory allocation is evaluated. The pre-existing input parameters such as the session vector ($V_{session}$) and the receiver's public key (PU_{rx}) are passed and thus are explicitly excluded from the auxiliary memory calculation.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
H_{enc} (Ciphertext Hash Sum)	Integer (Scalar)	1 × 1	c
SignatureTargetHash	Integer (Scalar)	1 × 1	c
i (Loop iterator)	Integer (Scalar)	1 × 1	c

Table 4 – Table method for considering space complexity of Algorithm 4

The total auxiliary space S represents the sum of dynamically allocated memory blocks during the vector dot product computation:

$$S = c + c + c$$

$$S = 3c$$

The highest memory usage allocated in the call stack is evaluated for rigorously establishing the closed-form space complexity. Let S_{prop} be the proposition stating that the total auxiliary memory allocated for executing the encapsulation process remains strictly constant. The variables H_{enc}, SignatureTargetHash, and i are the basic scalar primitives.

Running totals are repeatedly rewritten in-place within the same memory registers during the multiplicative encapsulation loop. The loop ends deterministically without scaling in relation to payload size as the dot product only works on a session vector that is theoretically limited to exactly three binary elements. There is no dynamic assignment of new memory addresses. As a result, the usage of scalar memory is strictly limited to the rigid constant 3c. The ultimate asymptotic bound is established by hypothetically eliminating the integer multiplier and the operational word coefficient c.

$$S = O(1)$$

5) Algorithm 5: Plaintext Cleaning and Format Extraction

Let N be the total character length of the input plaintext string (RawText). The raw input string is excluded from the auxiliary memory computation as it performs as pre-existing input space.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
CleanText	String Array	$1 \times N_{clean}$	$c \cdot N_{clean}$
MarkerMap	Object List	$1 \times N_{marks}$	$c \cdot N_{marks}$
CurrentChar	Char (Scalar)	1×1	c
NewMarker	Object (Scalar)	1×1	c
i (Loop Iterator)	Integer (Scalar)	1×1	c

Table 5 – Table method for considering space complexity of Algorithm 5

Let N_{clean} be the length of the extracted uppercase alphabetical characters, and N_{marks} be the total number of extracted formatting markers. The following initial space complexity equation is obtained from the table 5.

$$S(N) = c.N_{clean} + c.N_{marks} + 3c$$

The closed-form space complexity is strictly determined by evaluating the highest memory usage allocated in the call stack. Let $S_{prop}(N)$ be the proposition stating that the total memory used for formatting and cleaning a plaintext of starting length N scales absolutely linearly. In order to keep a constant usage of 3c, the variables CurrentChar, NewMarker, and the iterator i function as scalar primitives and are updated in-place during the evaluation loop. Memory is dynamically allocated by the MarkerMap data structure and the CleanText string in proportion to the number of characters assessed. The overall length of the processed input string ($N_{clean} + N_{marks} = N$) is always strictly equal to the sum of the alphabetical characters (N_{clean}) and the formatting markers (N_{marks}) mathematically.

The space is expanded to $S(N) = cN + 3c$ by substituting the absolute input length into the equation. The non-scaling scalar constant 3c and the operational word coefficient c are mathematically eliminated.

$$S(N) = O(N)$$

6) Algorithm 6: First Shift and Alphabet Substitution

Let N be the total character length of the cleaned input plaintext. The input string and the static EnochianMap dictionary are treated as pre-existing memory spaces.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
ShiftedText	String Array	1 × N	c . N
MappedArray	String List	1 × N	c . N
Temporary Loop Scalars	Mixed (Scalars)	1 × 5	5c

Table 6 – Table method for considering space complexity of Algorithm 6

Let N_{pad} be the total length of the array after structural padding is applied. The total auxiliary space S(N) represents the sum of dynamically allocated memory blocks during

$$S(N) = c.N + c.N + 5c$$

$$S(N) = 2cN + 5c$$

The peak memory usage allocated in the call stack is evaluated to strictly determine the closed-form space complexity. Let $S_{prop}(N)$ be the proposition stating that the total memory allocated for executing the homophonic substitution scales strictly linearly. The temporary scalar variables of CharIndex, ShiftedIndex, CurrentChar, DictionaryResult, and i are applied for arithmetic calculations and dictionary lookups while the two-stage process is in progress. These values are continually overwritten in place and their collective memory usage is restricted to the constant 5c.

The dynamic data structures named ShiftedText and MappedArray are generated by the algorithm. Both structures need contiguous memory allocation for exactly N elements as a one-to-one character evaluation basis is operated sequentially by the cryptographic process. Each array consumes c . N space and the highest-order scaling term is isolated so that the non-scaling scalar constant 5c and the operational word coefficients are dropped.

$$S(N) = O(N)$$

7) Algorithm 7: Second Shift and Matrix Allocation

Let N be the total element length of the shifted numerical array (MappedArray), and M be the defined matrix dimension. The initial one-dimensional array is applied as pre-existing input space.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
ValueList	List of Matrices	B × (M × M)	c . N
MarkerList	List of Matrices	B × (M × M)	c . N
Temp_Value_Matrix	Integer Matrix	M × M	c . M^2
Temp_Marker_Matrix	String Matrix	M × M	c . M^2
Scalar Variables	Mixed (Scalars)	1 × 8	8c

Table 7 – Table method for considering space complexity of Algorithm 7

The total auxiliary space S(N) is the sum of dynamically allocated memory structures needed for partitioning the entire array into discrete lists:

$$S(N) = 2cN + 2cM^2 + 8c$$

The peak memory usage allocated in the call stack is evaluated for establishing the architectural constraints to strictly establish the closed-form space complexity. Let $S_{prop}(N)$ be the proposition stating that the total memory allocated to shift and partition an array of length N scales strictly linearly. The algorithm requires temporary M × M matrices to format the data before appending them to the main lists. Since the system's architectural upper bound restricts matrix dimensions to $M \leq 10$, these temporary structures never go beyond 100 elements ($M^2 \leq 100$). This structural maximum constitutes an immutable, non-scaling constant memory block (C_{matrix}). The temporary loop scalars (Capacity, iterators, extractions) similarly remain bound to a constant 8c.

The ValueList and MarkerList data structures require dynamic memory allocation to store B discrete matrix blocks. Since the total number of elements partitioned across all combined matrix blocks mathematically equates to the original payload length N including zero-padding edge cases, each list strictly consumes c . N auxiliary space. The equation expands to $S(N) = 2cN + C_{matrix} + 8c$ by substituting the bounded matrix constant into the equation, The static structural bounds and scalar operational constants are eliminated by isolating the highest-order scaling term.

$$S(N) = O(N)$$

8) Algorithm 8: Chaotic Key Factor and Matrix Generation

The runtime execution environment's dynamic memory allocation is evaluated. The algorithm creates the session-specific key matrix by evaluating differential equations in continuous time.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
ValidatedKeyMatrix	Float Matrix	M × M	c . M^2
BaseMatrix	Float Matrix	M × M	c . M^2
ODE State Variables (x, y, z)	Float (Scalars)	1 × 3	3c
ODE Constants (α, γ, β)	Float (Scalars)	1 × 3	3c
Operational Scalars (x, y, z, etc.)	Mixed (Scalars)	1 × 6	6c

Table 8 – Table method for considering space complexity of Algorithm 8

The total auxiliary space S(M) is the sum of dynamically allocated memory structures during the generation and validation loops:

$$S(M) = 2cM^2 + 12c$$

To properly evaluated the closed-form space complexity, the call stack's peak memory usage is evaluated. Let $S_{prop}(M)$ be the statement that the total memory allocated resolves to a strict O(1) constant bound because of systemic dimensional limits. Determinants and iterators are instances of scalar primitives that are continuously updated in-place within the same memory registers for the Euler method iterations and mathematical validation. The scalar memory usage is tightly limited to the constant 12c since no new memory addresses are not dynamically allocated at each iteration step.

Contiguous dynamic memory allocation is required for the ValidatedKeyMatrix and BaseMatrix structures. This corresponds to a geometric $O(M^2)$ space complexity under unbounded theoretical assumptions. However, the matrix dimension is rigidly limited to a maximum boundary of 10 ($M \leq 10$) by the cryptosystem's core architecture. The maximum memory allocation for every matrix is limited to $M^2 = 100$ entries since $M \leq 10$. Regardless of the overall plaintext payload size, this structural maximum is an immutable constant C_{matrix} that never grows. The geometric scaling term $2cM^2$ collapses into an isolated, non-scaling constant when the capped constant is substituted into the closed-form equation.

$$S(M) = O(1)$$

9) Algorithm 9: Chaotic Seed Extraction and Assignment

The global session state is strictly assigned to pre-calculated scalar values by this algorithm. The algorithm treats the final floating-point Lorenz coordinates as an existing input space. The runtime execution environment's dynamic memory allocation is evaluated.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
Integer(Y) (S-Box Seed)	Integer (Scalar)	1 × 1	c
Integer(Z) (Hash Base)	Integer (Scalar)	1 × 1	c

Table 9 – Table method for considering space complexity of Algorithm 9

The total auxiliary space S is the sum of dynamically allocated memory blocks during the rounding and assignment process:

$$S = c + c$$

$$S = 2c$$

In order to rigorously establish the closed-form space complexity, the peak memory usage allotted in the call stack is evaluated. Let S_{prop} be the proposition stating that the algorithm allocates the total memory for extracting and assigning the chaotic seeds remains strictly constant. The fundamental rounding operations on pre-existing floating-point scalars are performed solely by the execution of the algorithm to produce the integer variables Integer(Y) and Integer(Z). These two scalar primitives are overwritten in-place within fixed memory registers and assigned to the global session state. The entire auxiliary memory usage evaluates to a strict, non-scaling constant of 2c since no dynamic arrays, lists, or matrices are generated and no iterative loops dependent on payload size are executed.

The absolute constant bound is established by analytically eliminating the static integer multiplier and the operational word coefficient c from the closed-form equation.

$$S = O(1)$$

10) Algorithm 10: Chaotic S-Box Execution

Let N be the payload's entire character length divided across the matrix blocks. The SBoxSeed(Y) is an isolated scalar input, and the input ValueList with the padded matrices is treated as pre-existing memory space.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
SubstitutedValueList	List of Matrices	B × (M × M)	c . N
SBox	Integer Array	1 × 21	21c
SubstitutedMatrix	Integer Matrix	M × M	c . M^2
Temporary Loop Scalars	Mixed (Scalars)	1 × 6	6c

Table 10 – Table method for considering space complexity of Algorithm 10

The total auxiliary space S(N) is the sum of dynamically allocated memory structures during the non-linear substitution loop:

$$S(N) = c.N + c.M^2 + 21c + 7c$$

$$S(N) = cN + cM^2 + 27c$$

In order to properly determine the closed-form space complexity, the call stack's peak memory usage is evaluated. Let $S_{prop}(N)$ be the proposition stating that the total memory used to replace an array of length N scales absolutely linearly. The SBox array is generated by the algorithm for mapping the scrambled permutations. There are 21 memory places needed for the array because the Enochian mathematical space is strictly limited to 21 characters. Regardless of the size of the payload, this array evaluates strictly to a 21c constant and never grows. The scalar primitives (PRNG, r,c OriginalValue, Rows, Cols) used for random generation and iteration are constantly rewritten in-place, limiting their combined usage to the constant 6c.

For every block, a temporary SubstitutedMatrix is made during the iterative substitution. This temporary structure never surpasses 100 integer elements ($M^2 \leq 100$) because of the systemic dimensional restriction of $M \leq 10$, creating a non-scaling memory block (C_{matrix}) that is continuously overwritten. To store the substituted matrices, dynamic memory allocation is required for the SubstitutedValueList. The encompassing list strictly uses c . N auxiliary space since the total number of integer elements kept across all merged blocks strictly equals exactly N.

By substituting the limited matrix constant, the following expanded equation is obtained:

$$S(N) = cN + C_{matrix} + 27c$$

The static structural bounds and scalar operational constants are mathematically eliminated.

$$S(N) = O(N)$$

11) Algorithm 11: Hill Cipher Matrix Multiplication

Let N be the total character length of the payload. The substituted matrices and the session key matrix are evaluated as pre-existing input spaces.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
EncryptedValueList	List of Matrices	B × (M × M)	c . N
EncryptedMatrix	Integer Matrix	M × M	c . M^2
Temporary Loop Scalars	Integer (Scalars)	1 × 4	4c

Table 11 – Table method for considering space complexity of Algorithm 11

The sum of dynamically allocated memory structures needed for performing linear multiplication across all blocks is the total auxiliary space S(N):

$$S(N) = c.N + cM^2 + 4c$$

The closed-form space complexity is objectively determined by evaluating the maximum memory output generated in the call stack at any given time during execution. Let $S_{prop}(N)$ be the proposition stating that the total memory allotted for multiplying a payload of initial length N scales absolutely linearly. The matrix dimension variables of r, c, rows, cols and iterators perform as fundamental scalar primitives. The scalar memory usage is absolutely limited to the constant 4c since these values are replaced in-place within the same memory registers.

An ephemeral EncryptedMatrix structure is constructed for all substituted matrix that have been substituted to perform the dot product results prior to the modulo arithmetic application. During the matrix multiplication, the mathematical massive amount of payload is maintained. As a result, the total number of elements in all ciphertext blocks is remained strictly N. Rigorously, the list uses c . N auxiliary space.

The operational word coefficient and all non-scaling structural constants are eliminated when the bounded matrix constant is substituted into the equation, isolating the highest-order scaling term.

$$S(N) = O(N)$$

12) Algorithm 12: Rolling Hash Tagging and Shuffle

Let N be the payload's entire character length divided across the encrypted matrix blocks. The HashBase(Z), the HashModulus, and the input EncryptedValueList with the ciphertext matrices are regarded as pre-existing memory regions. Let B be the total number of matrix blocks needed.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
Deck / ShuffledDeck	Array of Cards	$B \times (M^2 \times 1)$	$c \cdot N$
NewCard (Temporary)	Card Object	$M^2 + 1$	$c \cdot M^2$
CurrentHash / PreviousHash	Integer (Scalars)	1×2	$2c$
i (Loop Iterator)	Integer (Scalars)	1×1	c

Table 12 – Table method for considering space complexity of Algorithm 12

The total auxiliary space S(N) is the sum of dynamically allotted resource structures required for tagging and shuffling the ciphertext blocks:

$$S(N) = c.N + cM^2 + 3c$$

The highest memory usage create in the call stack is evaluated for strictly determining the closed-form space complexity. Let $S_{prop}(N)$ be the proposition stating that the total memory allocated for card tagging and shuffling a payload of initial length N scales strictly linearly. The fundamental scalar primitive variables for mathematical rolling hash progression are CurrentHash, PreviousHash,and the iterator i. They are overwritten within the same memory instantly which is maintaining strict constant memory usage of 3c.

A temporary NewCard object is constructed for encapsulating the current M × M matrix card along with the generated integer hash tag while the tagging loop is in progress. Since the cryptosystem's architecture mathematically limits the dimension of the matrix up to 10 × 10, There is no temporary matrix card that exceeds 101 elements ($M^2 + 1 \leq 101$). Thus, an immutable, non-scaling constant block (C_{card}) is formed from this structural maximum. Then, there are B card objects for storage that requires dynamical memory allocations for the Deck array. The whole data structure strictly uses c . N auxiliary space as the matrix data included within the cards strictly equals the exact mass of the entire payload N. By swapping memory pointers, the cryptographic

Fisher-Yates shuffling algorithm rearranges the array items immediately, requiring exactly zero additional geometric space.

The expanded space complexity form is obtained by substituting the bounded card constant into the initial equation:

$$S(N) = c.N + C_{card} + 3c$$

The structural bounds and scalar constants are eliminated by isolating the highest-order scaling term.

$$S(N) = O(N)$$

13) Algorithm 13: Subset-Sum Signature Generation

Let K be the architectural length of the sender's private key sequence. The sender private key (PR_{tx}) array and target header (H_{enc}) are passed as pre-existing input parameters and thus are not included in the auxiliary memory computation.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
V_{sig} (Signature Vector)	Binary Array	1 × K	c . K
TargetRemainder	Integer (Scalar)	1 × 1	c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 13 – Table method for considering space complexity of Algorithm 13

The sum of dynamically allocated memory blocks during the subset-sum trapdoor resolution is the total auxiliary space S(K) of the algorithm:

$$S(K) = c.K + c + c$$

$$S(K) = c.K + 2c$$

In order to determine the closed-form space complexity, the maximum memory output established in the call stack at any given point during execution is evaluated. Let $S_{prop}(K)$ be the proposition stating that when creating a digital signature using a private key of length K, the total memory allotted scales exactly linearly in proportion to K. The variables named TargetRemainder and the iterator i performs as the basic scalar primitives. While the greedy resolution loop process is in progress, the system continuously updates the remainder and overwrites within the same memory register instantly which depends upon the subset-sum subtractions. The total scalar

memory output is strictly limited to the constant 2c because no new memory is dynamically allocated per iteration.

The contiguous dynamic memory allocation is required for the V_{sig} data signature to construct the output signature vector. The signature array has to belong the exact same dimensional length K as the sender's private key vector for maintaining the mathematical alignment with the trapdoor subset. Each element within the vector takes a constant word size c to store a binary bits and hence, the array strictly allocates the c . K memory space. By separating the highest-order scaling term from the closed-form equation, the non-scaling scalar constant 2c and the operational word coefficient are mathematically removed.

$$S(K) = O(K)$$

14) Algorithm 14: Payload Serialization

Let N be the total mathematical mass of the encrypted ciphertext payload, and K be the architectural length of the digital signature vector. The individual components of header, signature, and shuffled deck are passed into the function as pre-existing memory blocks.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
TransmittablePayload	Data Structure	1 × (N + K + 1)	c . (N + K + 1)
SerializedData	Byte Array	1 × (N + K + 1)	c . (N + K + 1)

Table 14 – Table method for considering space complexity of Algorithm 14

The total auxiliary space S(N, K) is the sum of dynamically allocated memory structures required to concatenate and serialize the discrete cryptographic blocks for network transmission:

$$S(N,K) = c(N + K + 1) + c(N + K + 1)$$

$$S(N,K) = 2cN + 2cK + 2c$$

In roder to determine the closed-form space complexity, the highest memory usage allocated in the call stack is evaluated. Let S_{prop}(N, K) be the proposition stating that the total memory allocated for serialization scales strictly linearly in proportion to the combined mass of the payload and signature. The TransmittablePayload container is created by the algorithm to put the header, signature vector, and ciphertext deck into it together. After that aggregation, that container is mapped into a contiguous byte stream SerializedData by the Serialize function that is suitable for network routing such as JSON or XML formats. Contiguous dynamic memory

allocation that is directly proportional to the total size of the N payload elements and the K signature elements is required for both procedures.

The non-scaling operational scalar constants are eliminated in the asymptotic complexity, Moreover, since the overall ciphertext payload N mathematically dominates the fixed-length asymmetric key bounds K ($N \gg K$). The key length is absorbed as a lower-order term.

$$S(N) = O(N)$$

15) Algorithm 15: Digital Signature Verification

The runtime execution environment's dynamic memory allocation is evaluated. Since they make up pre-existing input space, all vectors and cryptographic parameters passed from the received packet such as the header, signature, public key, modulus, and multiplier are strictly excluded from the auxiliary memory computation.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
Checksum	Integer (Scalar)	1 × 1	c
Inv_N	Integer (Scalar)	1 × 1	c
VerificationResult	Integer (Scalar)	1 × 1	c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 15 – Table method for considering space complexity of Algorithm 15

The sum of dynamically allocated memory blocks during the authentication sequence is the total auxiliary space S:

$$S = c + c + c + c$$

$$S = 4c$$

The highest memory usage established in the call stack at any given point during execution is evaluated for strictly determining the closed-form space complexity. Let S_{prop} be the proposition stating that the total memory allocated by the algorithm to verify a digital signature evaluates to a strict constant bound. The variables named Checksum, Inv_N, VerificationResult, and the loop iterator i serves as the fundamental scalar primitives. The Checksum is iteratively overwritten instantly within the same memory register during the dot product calculation loop. Without starting

any dynamic array or matrix generation, the subsequent modular inverse transformation allocates the computed integer results to isolated variables.

Regardless of the overall transmission size, the memory usage is always limited to the constant 4c since the method rigorously calculates scalar values based on fixed-length cryptographic keys. The final asymptotic bound is established by dropping the operational word coefficient.

$$S = O(1)$$

16) Algorithm 16: Subset-Sum Key Decapsulation

Let K be the receiver's private key sequence's architectural length. The runtime execution environment's dynamic memory allocation is evaluated. The function receives the received header (H_{enc}) and the receiver's private cryptographic parameters (PR_{rx}, M_{rx}, N_{rx}) as pre-existing memory spaces and they are not included in the auxiliary memory computation.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
$V_{session}$ (Session Vector)	Binary Array	1 × K	c . K
Inv_N / TargetSum	Integer (Scalars)	1 × 2	2c
CurrentSum (Remainder)	Integer (Scalar)	1 × 1	c
i (Loop Iterator)	Integer (Scalar)	1 × 1	c

Table 16 – Table method for considering space complexity of Algorithm 16

The sum of dynamically allocated memory blocks required to solve the knapsack trapdoor is the total auxiliary space S(K):

$$S(K) = c.K + 2c + c + c$$

$$S(K) = c.K + 4c$$

In order to rigorously determine the closed-form space complexity, the call stack's memory usage is evaluated. Let $S_{prop}(K)$ be the total memory used to solve the trapdoor and extract the session vector scales absolutely linearly in proportion to K. Basic scalar primitives include the variables Inv_N, TargetSum, CurrentSum, and the loop iterator i. The CurrentSum remainder is continually updated and rewritten instantly within the same memory register based on subset-sum subtractions while the greedy resolution loop is running. The overall scalar memory usage

evaluates to a strict constant bound of 4c as additional memory addresses are never dynamically created every iteration step.

For the $V_{session}$ array to hold the decapsulated binary bits, the technique necessitates contiguous dynamic memory allocation. This array is started with the exact dimensional length K of the private key vector in order to preserve mathematical alignment with the trapdoor. Every element takes up a fixed word size c. As a result, the array output strictly uses a c . K auxiliary space. The non-scaling scalar constant 4c and the operational word coefficient are logically removed by isolating the highest-order scaling component from the closed-form equation.

$$S(K) = O(K)$$

17) Algorithm 17: Payload Sorting and Key Regeneration

Let N be the payload's total character length divided across the ciphertext matrices. Let B be the total number of matrix cards in the shuffled deck, which may be expressed mathematically as $B = \frac{N}{M^2}$. As pre-existing input spaces, the ShuffledDeck and the isolated scalar parameters (HashModulus, KeyFactor, BaseMatrix, HashBase) are evaluated.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
ExpectedHashes	Integer Array	1 × B	c . B
Introsort Call Stack	Memory Stack	$\approx log_2(B)$	c . log B
ValidatedKeyMatrix	Float Matrix	M × M	c . M^2
Operational Loop Scalars	Mixed (Scalars)	1 × 5	5c

Table 17 – Table method for considering space complexity of Algorithm 17

The sum of the dynamically allocated memory structures required to reconstruct the sequence and regenerate the key matrix is the total auxiliary space S(N). It is important to note that B is scaled in proportion to N:

$$S(N) = c.B + c.logB + cM^2 + 5c$$

The closed-form space complexity is exactly determined by evaluating the maximum memory output generated in the call stack at any given time during execution. Let $S_{prop}(N)$ be the statement that the total memory used to regenerate the session matrix and sort the deck scales strictly linearly. Two primary space-consuming subroutines are performed by the algorithm. In

order to map the correct mathematical sequence, it first creates the ExpectedHashes arrays. This arrays requires contiguous dynamic memory for exactly B entries because each card has a unique tag. The Introsort function is then carried out by the algorithm. Introsort makes extensive use of recursion even though it does the physical sorting instantly without the requirement of additional geometric array space. Before switching to Heap Sort, the algorithm theoretically imposes a recursion depth restriction of $2 \log_2 B$. Thus, the auxiliary memory space required to maintain the local variables in the call stack is exactly O(log B).

The ValidatedKeyMatrix is regenerated in the fourth stage. The maximum dynamic allocation for this matrix is firmly controlled at 100 elements since the cryptosystem architecture requires that the matrix dimension strictly cannot exceed 10 ($M \leq 10$). This creates an unchangeable constant block (C_{matrix}), which is independent on the payload length N. The boundary constants are substituted to produce the extended equation: $S(N) = cB + c\ log\ B + C_{matrix} + 5c$. The logarithmic term c log B is mathematically dominated to the linear term cB due to the direct correlation between B and N. To isolate the highest-order scaling bound, the dominated terms and non-scaling constants are eliminated.

$$S(N) = O(N)$$

18) Algorithm 18: Decryption and Reverse Substitution

Let N be the payload's total character length divided across the ciphertext matrices. Let B be the total number of matrix cards, which may be expressed mathematically as $B = \frac{N}{M^2}$. The scalar substitution seed (SBoxSeed(Y)), the session key matrix (ValidatedKeyMatrix), and the sorted deck (SortedDeck) are supplied into the function as pre-existing memory spaces and are not included in the auxiliary memory computation.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
RecoveredValueList	List of Matrices	B × (M × M)	c . N
SBox	Integer Array	1 × 21	21c
Temporary Matrices	Integer Matrices	4 × (M × M)	4c . M^2
Operational Scalars	Mixed (Scalars)	1 × 7	7c

Table 18 – Table method for considering space complexity of Algorithm 18

The sum of the dynamically created memory structures needed to perform inverse linear algebra and unscramble the Enochian mapping is represented by the entire auxiliary space S(N):

$$S(N) = c.N + 4cM^2 + 28c$$

The closed-form space complexity is exactly determined by evaluating the maximum memory output generated in the call stack at any given time during execution. Let $S_{prop}(N)$ be the proposition stating that the total memory used to reverse the matrix encryption on a payload of initial length N scales absolutely linearly. The arithmetic variables (Det, DetInv, Rows, Cols, r, and c) and iterators function as basic scalar primitives. The scalar memory usage is absolutely limited to the constant 7c since these values are substituted instantly within the same memory registers.

In order to map the reverse chaotic permutations, the algorithm creates the SBox array. This array requires exactly 21 memory blocks since the Enochian mathematical space is strictly limited to 21 letters. This structure evaluates strictly to a non-scaling constant 21c and never grows, regardless of the amount of the payload. Four temporary mathematical matrices named AdjugateK, KeyMatrixInverse, E_{matrix}, and V_{matrix} are created during the inverse computation and multiplication cycles. Each temporary allocation is strictly limited to 100 elements ($M^2 \leq 100$) due to matrix dimensions being structurally fixed to a maximum limit of 10 × 10. Rigid non-scaling constant memory chunks (C_{matrix}) are what these structures function as.

To store the decrypted outputs, dynamic memory allocation is needed for the final RecoveredValueList. During reverse substitution, the payload's mathematical mass is carefully maintained. As a result, the total number of elements in all restored blocks stays exactly N. The c . N auxiliary space is exclusively used by the overall list. The operative word coefficients and other non-scaling structural constants are eliminated by substituting the equation with the bounded matrix constant, which isolates the highest-order scaling term.

$$S(N) = O(N)$$

19) Algorithm 19: Remapping and Plaintext Assembly

Let N be the decrypted payload's total element length. Homophonic dictionaries, shift parameters (C_1, C_2), recovered matrices (RecoveredValueList), and archived formmating lists (MarkerList, CharMarkerMap) are all handled as pre-existing input spaces.

Variable / Data Structure	Data Type	Dimensionality	Memory Cost
ShiftedEnochianArray	Integer Array	1 × N	c . N
ShiftedEnglishString	String Array	1 × N	c . N
CleanPlaintext	String Array	1 × N	c . N
FinalPlaintextString	String Array	1 × N	c . N
Operational Scalars	Mixed (Scalars)	1 × 8	8c

Table 19 – Table method for considering space complexity of Algorithm 19

The total auxiliary space S(N) is the sum of dynamically allocated memory structures required to reassemble the sequential plaintext string:

$$S(N) = 4cN + 8c$$

To properly determine the closed-form space complexity, the call stack's peak memory usage is evaluated. Let $S_{prop}(N)$ be the proposition stating that the memory used to unshift and combine the final plaintext sequence grows absolutely linearly. Basic scalar primitives are represented by the variables for dictionary searches, loop iterators, and arithmetic extractions. These variables are always updated instantly within the same memory registers throughout the multi-stage decrypting loops, maintaining a strictly constant memory usage of 8c.

Four main data structures must be generated sequentially according to the algorithm: The recovered matrices are flattened using ShiftedEnochianArray, the homophonic translated characters are stored in ShiftedEnglishString, the unshifted alphabetical sequence is stored in CleanPlaintext, and the final grammatical reconstruction with reinserted markers is stored in FinalPlaintextString. Each of these arrays and strings creates contiguous dynamic memory proportionally proportionate to the total plaintext length N since the decoding operations function strictly sequentially on a one-to-one evaluation scale. Every structure strictly uses c . N auxiliary space.

The absolute linear bound is established by analytically eliminating the additive scaling coefficients and the static operational scalar constants by separating the highest-order scaling term from the closed-form equation.

$$S(N) = O(N)$$